Praxis: an agentic vision coach for upper-limb rehab at home

Technical report · OpenCV AI Competition 2026 (powered by AWS) · Team Praxis · 24 Oct 2026

Live (AWS Graviton4 + COOL, HTTPS): **https://d235zashbh751z.cloudfront.net** · Judge quick-start: `docs/JUDGE_QUICKSTART.md` · Diagrams: `docs/ARCHITECTURE.md` · Full engineering detail, module by module: `docs/ENGINEERING_NOTES.md`

> Praxis helps people keep up with exercises a therapist has already prescribed, and coaches them while they do them. **It is not a medical device.** It does not diagnose and it does not choose exercises. Its angles and times are not clinical measurements.

Summary

After a stroke or a shoulder injury, most of the work of recovery happens at home, between clinic visits, with nobody watching. Patients get a printed sheet of exercises. They don't know if a repetition counted. They compensate without noticing. The therapist sees nothing until the next visit.

Praxis uses **one ordinary laptop camera, a printed A4 sheet and a cup or a few pegs** to coach three therapist-prescribed exercises:

- **Arm out to the side** (shoulder abduction, lifting the arm out sideways)
- **Waiter – cup** (a GRASP coordination task)
- the **Nine-Hole Peg Test (NHPT)** (a standardized weekly assessment)

How it works, briefly:

- **OpenCV 5 does the measuring.** The printed sheet carries four ArUco markers (square codes the camera can find and tell apart). From them we get a metric homography, which is the mapping from the camera picture to the flat sheet in millimetres. We find pegs, holes and the cup with Hough circles, HSV colour segmentation, contours and masked template matching. Hand and body landmarks (key points such as fingertips, shoulders and hips) come from OpenCV Zoo ONNX models run through `cv2.dnn`. Each exercise has a state machine that turns this into timestamped events: a rep, a reach, a peg seated, a trunk lean, tracking lost.
- **Stop rules in code run before any coach.** They cover pain ≥ 7, new symptoms, repeated trunk lean, repeated reps the camera could not check, sustained exertion and tracking lost. (New symptoms is still in the code and API, but since 21 Oct the patient uses Pain / feel unwell or Take a rest instead of a check-in box.) A stop rule pauses or stops the session and flags the therapist.
- **Two coaches share one interface.** A deterministic rule coach gives every cue during a rep. A **Claude agent coach** runs at checkpoints and calls OpenCV itself through MCP tools (MCP is a standard protocol for letting a model call tools). `check_setup` runs a fresh ArUco/homography/pose check on the latest frame. `resample_target_colour` re-segments a cup the camera cannot see. The camera result decides the agent's next step: name the covered

corner, re-sample the colour, or offer to resume, and it only offers to resume after the camera check passes. Every dose change and every resume is a prompt the patient answers.
- **It runs on AWS.** An EC2 **c8g.large (Graviton4)** launched from the **COOL AMI** runs the same service on COOL's OpenCV 5.1.0-dev behind CloudFront HTTPS. We measured COOL against the stock OpenCV wheel on the same instance.

What is measured and what is not, in one place:

Area	Status (25 Oct 2026)
OpenCV 5 pipeline, three exercises, every screen, stop rules, rule coach, trace, replay	built; 937 service tests (25 Oct) and 1019 web tests (24 Oct) pass; CI on every push
Claude agent coach (17 tools over MCP, autonomy table, guardrails)	built; Claude Sonnet 5.5 via the Anthropic API. **On real video** (REHAB24-6 run through the product pipeline, 40 sessions): a fault confirmed by motion capture was corrected in 41/44 set reviews, the fix was named on 26/26 real wrong-view pauses, **0 guardrail violations**. Synthetic, 15 scenarios, 21 Oct recheck: 15/15, 0 violations (§8.3)
AWS: Graviton4 + COOL + CloudFront	**live** (rebuilt 21 Oct; stopped between work sessions, started for judging from submission through 10 Nov). On 11 Oct, on the previous instance (same AMI and type), the test suite passed 634/636 on COOL and we measured the benchmark (§7). We checked the 21 Oct rebuild with `/health`, a replay and 3 agent runs
Accuracy on real people	**Arm out to the side only, on a public lab dataset** (REHAB24-6, 8 healthy adults, front camera, 88 reps): angle MAE **3.8°** vs motion capture (95 % limits −9° to +10°), rep-peak MAE 3.8°, rep F1 **0.93** (§8.4, after the 12 Oct threshold changes). It exposed two pose defects, both now fixed. NHPT and Waiter – cup: synthetic only (§8.1). Our own-recordings protocol is built but not run (§8.6).
Usability (SUS), patient or therapist pilots	not run. A team shake-down on the hosted build did run (15 live sessions, 4 testers including automated agent-driven sessions). It is not a usability study (§8.7)
S3, CDK	not built (§7.4). The agent runs on Claude via the Anthropic API; Bedrock is not used
CloudWatch logs, EMF metrics, dashboard, alarms	built and tested locally, not yet run on AWS (§7.5)

---

1. Problem

Stroke is common, disabling and growing. Stroke is the second-leading cause of death worldwide. If you count death and disability together, it is third. Between 1990 and 2019, new strokes rose 70 % and the number of people living after a stroke rose 102 % [1]. More than 70 % of people who have a stroke have upper-limb deficits (problems with the shoulder, arm or hand) [2]. In a group of 102 patients with arm paresis (weakness) after an acute stroke, only 11.6 % had fully regained use of the arm at six months [3]. Shoulder problems add to this. An ultrasound screening of a whole village found a full-thickness rotator-cuff tear in 22.1 % of residents, rising to 36.6 % of those in their 80s [4].

Recovery needs repetitions that clinic time does not give. In 312 observed therapy sessions, task-specific arm practice happened in only 51 % of sessions. When it did happen, it averaged **32 repetitions** [5]. The rest is meant to happen at home. But non-adherence to self-managed home physical therapy (people not doing it) can reach **70 %** [6]. Access is the other limit. About **2.4 billion** people live with a condition that could benefit from rehabilitation [7, 8]. In some low- and middle-income countries more

than half of them do not get the services they need, and many places have fewer than 10 skilled practitioners per million people [7].

Home programs work when people do them. GRASP (Graded Repetitive Arm Supplementary Program, UBC) is an arm program patients do on their own. In a randomized trial of 103 stroke inpatients, it improved arm function on the CAHAI (an arm and hand function scale) by 6.2 points over an education control group (95 % CI 3.4–9.0), and the gain held at five months [2]. The **Nine-Hole Peg Test** is a standard, quick measure of finger dexterity, with adult norms [9, 10]. In stroke, its smallest real difference for the more-affected hand is large: 32.8 s [11]. That is one reason a weekly test, given the same way each time and with per-peg splits, is more useful than an occasional clinic reading.

What goes wrong at home is specific.

1. *No feedback on quality.* The patient can't tell if the lift reached the target, if it was held, or if the trunk leaned to fake the range.
2. *No dose control.* On a bad day patients either push through pain or skip the session, and nobody sees either.
3. *No signal to the therapist.* The clinician hears about a missed week, a pain spike or a compensation pattern at the next visit, weeks later.
4. *Outcome measures need the clinic.* The Nine-Hole Peg Test is quick, but a clinician times it with a stopwatch.

Why a camera, and why now. Every patient with a laptop already owns the sensor. With four ArUco markers, OpenCV 5 can turn a printed sheet into a calibrated measuring surface. It can run hand and pose networks on a laptop CPU in milliseconds, all in memory, without storing video. What was missing is a system that is honest about what one webcam can measure, keeps the therapist in charge, and stops itself when it should.

2. Users and design principles

	Patient (primary)	**Therapist (secondary)**
Who	Adult after stroke or a shoulder injury, practising at home between visits. Often older, often using one hand, may have low vision or reduced dexterity. Not technical	Occupational therapist or physiotherapist with a caseload of home programs
Needs	Know what to do and whether it counted; a safe dose on bad days; an easy way to stop; no feeling of being watched	Adherence they can trust; early word of pain and safety events; an objective trend (NHPT time, reach time, range); control of the prescription
In Praxis	Check-in → demonstration → camera setup → live session with captions → 3-question feedback → report	Read-only dashboard: adherence from real sessions only, flags with reasons, NHPT trend, and the prescription editor (practice days, versioned)

Principles we stuck to (from the hi-fi wireframes, `docs/wireframes/`):

1. **The therapist prescribes, the coach proposes inside the prescription, and code enforces it.** Dose, allowed adaptations and stop rules live in a versioned prescription. Every proposed change is clamped to it (cut back to fit its limits).
2. **Unverified is not failed.** If the camera could not see a rep (the hand covers the cup, a marker drops out), the rep is counted as *unverified*. That count is shown next to the verified count everywhere and is never held against the patient.

3. **Feeling is self-reported.** Pain and effort come from what the patient taps or says. We never analyse the face for pain or emotion. Person detection is used for presence and pose only.
4. **No composite score.** No "accuracy" or "recovery" number appears anywhere. The app shows reps (verified / unverified), times and camera-relative angles, and labels each one.
5. **The patient can always stop.** Take a rest (which pauses) · Pain / feel unwell · End session are fixed on every live screen. Only the patient's tap resumes a session.
6. **Say what is simulated.** The rule coach says it is rule-based. A replay says "Replay". Sample data says "Sample" (and is off by default since 24 Oct, so a fresh install starts from zero). Every trace row names its source. On Home, the next-practice card is marked "Start here".

3. What exists and what is new

Camera and sensor rehab products already exist. This is how their makers describe them (vendor claims, which we have not checked ourselves):

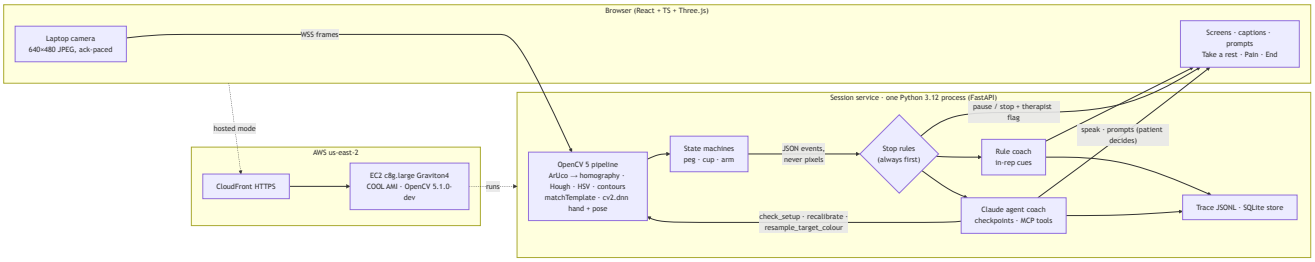
Product	Approach	How Praxis differs
Kemtai	software-only motion tracking with a phone, tablet or laptop camera; general exercise and musculoskeletal programs	Praxis adds a **metric printed surface** (ArUco homography) for hand and object tasks, a standardized dexterity test, and therapist stop rules in code
Neofect Smart Glove	wearable sensor glove with games for stroke hand and arm rehab	Praxis needs **no wearable**: one laptop camera, a printed sheet, a cup and pegs
MIRA	exergame platform for upper-limb rehab (feasibility study with 10 outpatients)	Praxis coaches the prescribed exercise itself (GRASP task, abduction) and **verifies each rep** from vision events instead of scoring a game

We searched and found no webcam product that times the Nine-Hole Peg Test at home and also coaches therapist-prescribed upper-limb exercises with an agent. That is only a negative search result. It does not prove that none exists.

What is new in Praxis:

- **A printed sheet as a measuring tool.** Four ArUco markers turn an A4 page into a calibrated plane (px ↔ mm, with the reprojection error reported). This is what lets a laptop camera time a Nine-Hole Peg Test peg by peg and measure cup reaches dot to dot, with no wearable.
- **An agent whose tools are OpenCV.** The agent does more than read events. It can re-run calibration on the current frame, re-sample the target's colour, and re-check the camera before it offers to resume. The vision result decides its next tool call (§6.3).
- **Safety outside the model.** Stop rules, clamps and the autonomy table are code, and the model cannot talk its way past them. The tests show this: On the shipped model (Claude Sonnet 5.5): 0 guardrail violations over the 15 synthetic scenarios, including a prompt-injection one, and 40 real-video sessions.
- **Honest perception.** Verified, unverified and missed are separate outcomes, each with a reason. Tracking loss pauses the session with the reason on screen, and the session resumes by itself when tracking comes back.

4. System architecture



One process, and it sends events, not frames. The browser sends 640 × 480 JPEGs over a WebSocket, each tagged with its capture time. It only sends the next frame after the service has answered the last one. That way a slow host gets fewer, fresher frames instead of a growing queue. The service decodes each frame with `cv2.imdecode` and runs the pipeline in a thread pool (OpenCV releases Python's GIL lock while it works). It sends back JSON: events, overlay geometry in frame pixels, captions and prompts. Frames are thrown away after processing. **No frame, pixel or image is written to disk, put in an event or a trace row, or sent to the model.** A test checks every agent tool result for pixels.

Two deployment modes, same code.

- *Edge mode* runs the service on the patient's laptop (`uvicorn`). No frame leaves the machine. This is our privacy story.
- *Hosted mode* runs the identical service on the c8g instance behind CloudFront (§7). Frames travel over TLS, are processed in memory and are dropped.

The camera-setup screen says where processing happens. **Replay mode** pushes a recorded clip through the same pipeline and coach with no webcam. It is the judges' fallback and is badged "Replay" everywhere.

Stack. Python 3.12, FastAPI, `opencv-python-headless==5.0.0.93` (pinned, for local runs; on AWS the service uses the COOL AMI's own OpenCV 5.1.0-dev build), NumPy, Strands Agents SDK + FastMCP 3.4 (MCP streamable HTTP, spec 2025-06-18), SQLite. React 19 + TypeScript + Vite, Radix, Tailwind 4, Three.js (a Blender-built GLB avatar, generated in code). Every dependency is pinned (`uv.lock`, `pnpm-lock.yaml`), and the ONNX models are pinned by SHA-256 hash.

5. OpenCV 5 implementation

Every perception module names `cv2.__version__` in its docstring. Each stage below is real OpenCV work. Each is tested on synthetic frames from our own renderer (`perception/synth.py`: pinhole camera, tilt, yaw, noise).

Stage	OpenCV 5	Output
Sheet calibration	`cv2.aruco.ArucoDetector` (DICT_4X4_50, sub-pixel corners) → `cv2.findHomography` on 16 corners; `cv2.RANSAC` fallback with 3 markers (threshold in px); `cv2.flip` re-detection for mirrored video; `cv2.solvePnP` (IPPE) + `cv2.Rodrigues` for camera tilt	px ↔ mm homography, reprojection RMS (≈ 0.2–0.4 mm on renders), source `fresh / partial / held`, `mirrored`

Stage	OpenCV 5	Output
Setup quality	luma mean (brightness), `cv2.Laplacian` variance (blur), marker-corner jitter over 10 frames, task-area projection, px/mm. Each missing marker is predicted from the visible ones (`cv2.findHomography`) and tested against the frame and ArUco's undecoded candidates	checklist rows, each with a specific fix: a corner is "outside the picture", "covered", or "in view but cannot be read"
NHPT board	`cv2.warpPerspective` to a 3 px/mm top view; normalisation against the paper (grey `morphologyEx` close → `cv2.divide`, which flattens uneven light and white-balances in one go, ≈ 0.5 ms board / 1.3 ms cup sheet); `cv2.HoughCircles` once (cached, snapped to the spec); `cv2.inRange` (HSV, skin band excluded for every skin tone) + `cv2.findContours` + `cv2.minAreaRect` length gate; masked `cv2.matchTemplate` (`TM_CCOEFF_NORMED`) as tie-breaker	`hole_k: empty / occupied`, free pegs in mm
Cup on target sheet	warp to 2 px/mm; the same paper normalisation; HSV `inRange` + `morphologyEx` open; contours grouped around the largest one (fingers split the base) and gated on size and `minAreaRect` aspect; `cv2.moments`	cup-base centre point in mm, over-dot test
Hand	OpenCV Zoo palm detector + hand-landmark ONNX via `cv2.dnn` (Apache-2.0)	21 landmarks → mm via the homography; pinch aperture, path speed, SPARC smoothness, tremor (FFT, 3–12 Hz)
Pose	OpenCV Zoo person detector + BlazePose ONNX via `cv2.dnn`	shoulder angle (One-Euro filtered), trunk lean, shoulder hike, view and visibility gates
Tap-to-sample colour	the patient taps the live picture (07b, 07) → frame px → sheet mm via the homography → 6 mm HSV patch on the warped sheet → ±12° hue window. Colourless patches (median S < 60) are refused, with the reason	works with wooden pegs or any saturated cup colour; never black

Calibration that copes with a real table. The first fix needs all four markers. After that, a forearm over a corner is normal. So three markers give a RANSAC homography, and fewer than three hold the last fix for 1.5 s. Only after that is the frame uncalibrated. Twenty such frames raise `perception_lost`, which pauses the session with the reason on screen. `perception_regained` resumes it.

The sheet homography ignores height, so `solvePnP` estimates the camera's tilt from the marker corners. For a laptop camera tilted forward over a sheet 25–35 cm away, setup advises staying below 50°. The 50° cap comes from geometry (the top of a 20 mm peg must not land on the next hole at 32 mm pitch). It is unverified on real pegs.

Tested against the room, not only the render (11 Oct, synthetic sweep).
`evaluation/scripts/robustness.py` renders both sheets under 107 perturbations, one at a time and combined, and runs the real calibration, setup rows, occupancy and cup code (`evaluation/results/robustness_v0.md`). Its first run found real weak spots. We fixed each one test-first:

- A dim room lost every peg and the cup. Fix: normalise each top-down view to the paper's own brightness, using a grey-level closing that flattens the light and white-balances in one step.
- A warm bulb turned the paper into 6–7 phantom pegs. Same fix.
- A deep skin tone read as four pegs. The skin band now reaches V 40.

- A pen read as a peg, and as a cup 85 mm away. We added length and aspect gates.
- Fingers splitting the cup base hid a cup that was 55 % visible. Fragments are now grouped.
- Mirrored video never calibrated. ArUco now re-runs on the flipped frame and maps the corners back.
- The A4 PDF printed on US Letter lost two markers. The page is now cut to the area both paper sizes share, with the geometry unchanged.

After the fixes, on renders, the sheet calibrates:

- from 0° to 65° tilt, at any in-plane rotation, and from 30 % to 100 % frame fill;
- at mean luma 30, under warm and cool colour casts, shadows, noise σ 20, JPEG q40, 5 px motion blur and defocus r3.

The worst target error is 1.2 mm (a sheet filling 30 % of the frame). The exceptions are a curled or creased sheet (1.2–11 mm) and a barrel lens (1–2.4 mm, because the homography assumes no lens distortion). Every setup row in the sweep names the right cause. A code review also fixed two timing biases (§8.1) and a units bug: the three-marker RANSAC threshold was 3 mm, not the documented 3 px.

Three state machines turn geometry into events.

- *Nine-Hole Peg Test*: `at_source → grasped → in_transit → over_target → seated → hand_clear`. Grasp fires when a free peg disappears, or when the pinch aperture is ≤ 25 mm in the source zone (pegs lifted from a dish). Occupancy is debounced over 0.25 s (it must hold that long to count). If a hole fills with no hand within 30 mm in the last 0.5 s, it is counted but marked `unverified (no_hand_near_hole)`. The timer runs from the first lift to the ninth seat, with per-peg splits. Events are `rep_complete`, `wrong_hole` and `drop`.
- *Waiter – cup*: `at_dot → lifted → in_transit → over_dot → placed → hand_clear` along the arc 1→5→1, with reach time per dot and `wrong_dot`. Two slow arcs come first. The two fast arcs come **only after the patient answers Yes** to the progression prompt. A placement seen only through the hand is `unverified (cup_occluded)`, unless the base was seen resting on the target dot within 0.6 s before hand-clear.
- *Arm out to the side*: `start_pose → moving → target_reached → hold → return`. The target is clamped to the 90° ceiling in code. A verified rep reaches target − 10° and holds 3 s. Misses (`target_missed`, `hold_too_short`) are counted apart from unverified reps.
    - The machine freezes (`arm.gate`) when the patient faces the wrong way or the joints are not visible, so it never scores what it cannot see.
    - If the gate stays shut for 3 s (turned away from the camera) or 6 s (the practising shoulder, elbow or hip not visible), the set pauses with the fix (`setup_blocked`, carrying the visibility numbers). The 6 s was 3 s until the 21 Oct team shake-down. There, every arm setup pause was this check, and most were short dips in the middle of a set (`team_test_2026-10-21.md`).
    - `compensation_lean` needs |lean| > 12° held 0.5 s with both hips visible.
    - `shoulder_hiking` needs both shoulders visible and shoulder elevation above the healthy allowance *at the current arm angle* (REHAB24-6 p95 + 0.05 trunk lengths, §8.4) for 3 frames.
    - The front view is accepted from a shoulder separation of 0.55 trunk lengths (about 20–30° of turn, §8.4).

Exertion is behavioural, and labelled that way. Rep-time slope, falling peak angles, tremor share and the last self-reported effort (which fades after 120 s of steady work) give `low / moderate / high` with

plain-language reasons. The chip in the session and on the report says *behavioural*. Sixty seconds at high is a stop rule. We do not estimate heart rate from the face.

The avatar is a reference, never a judge. It is a gender-neutral mannequin, built in code by headless Blender (no third-party meshes) and played with Three.js `AnimationMixer`. It demonstrates each exercise and shows the tempo (lift 3 s, hold, lower) on the live screen. Recorded human motion (UI-PRMD, IntelliRehabDS) shapes two clips, scaled so the arm never goes above the prescribed 90°. Reps are verified from OpenCV events only.

6. Coaching: stop rules first, then two coaches

6.1 Order of operations per frame

perception events → behavioural exertion → **stop rules** → coach → trace.

The stop rules (`stop_rules.py`) take their thresholds from the prescription:

- pain ≥ 7
- new or worse symptoms at check-in
- trunk lean > 12° three times
- three reps in a row the camera could not verify
- three performance misses in a row (gentle rest)
- exertion high for 60 s
- tracking lost

They run before either coach sees the event. Each one writes a therapist flag where a clinician should see it. While a session is stopped or paused, both coaches stay silent.

6.2 Rule coach (every in-rep cue)

The rule coach is deterministic and rate-limited. It is labelled "Rule-based coach" on screen and in every trace line. Its actions are `instruct | correct | progress | regress | rest | stop`, and each `Decision` must cite the event ids it came from. It goes from event to cue in about 1 ms (§8.2), which is why in-rep cues stay with it.

The shoulder-hike caption is rationed to at most 2 per set, 20 s apart. We did this because in the 21 Oct team shake-down the flag fired on most reps (`team_test_2026-10-21.md`). The `shoulder_hiking` event itself is unchanged and is still traced and counted in the report.

6.3 Claude agent coach (checkpoints)

At checkpoints (check-in, end of a set or arc, a stop-rule pause, NHPT complete, session end), `AgentCoach` runs one background task with Claude. We use Sonnet 5.5 via the Anthropic API since 12 Oct, picked in a measured comparison (§8.3). Amazon Bedrock is not used. The fixed system prompt and tool definitions are prompt-cached (91–94 % of input tokens are read from cache). Each task has limits: ≤ 6 tool calls, 15 s, 12 runs per session, and a daily cap. The model can reach **17 tools on three FastMCP servers** (`perception`, `exertion`, `coach`) over streamable HTTP. If MCP is down, the same functions are called directly and the trace says `transport: direct`.

OpenCV tools, not only event readers.

- `check_setup` runs, on the latest frame (held in memory by reference): a fresh `cv2.aruco` detection, `cv2.findHomography` (reprojection error in mm), `cv2.solvePnP` tilt, lighting, sharpness, marker steadiness and target visibility.
- `recalibrate` resets the calibrator or pose gate and reports the next frames.
- `resample_target_colour` warps the sheet, samples HSV where the target should be, and applies the new colour window **only if `inRange` + `findContours` then find the target where they failed before**.

OpenCV output changes the next action. This is from the real-Claude trace `traces/agent/anthropic/covered_marker.jsonl` (11 Oct, Claude Sonnet 5; the 21 Oct Sonnet 5.5 run of the same scenario is in `traces/agent/anthropic-sonnet-5-5/`):

```
 stop rule    failed_reps: "Three in a row I couldn't verify. Let's pause and check the
setup."
agent tool   get_set_summary
agent tool   check_setup → cv2.aruco: markers [0,1] found, [2,3] missing; "The markers in
the
                    bottom-right corner (2), bottom-left corner (3) are covered or creased
…"   (5 ms, MCP)
agent tool   speak "Your reps weren't seen because two corner markers are covered. Please
clear
                    and flatten the bottom-right and bottom-left corner markers on the
sheet."
agent tool   recalibrate → markers_found [0,1,2,3], rms_mm 0.165, tilt_deg 15.9      (102
ms)
agent tool   request_resume → fresh check_setup passed → prompt "… Ready to carry on?"
patient      Yes, resume → "Thanks. Carrying on from where we stopped."
```

The scenario covers two corners, the way a forearm resting across the bottom edge would. One covered corner is tolerated live (§5), so it never makes a reach unverified.

In `green_cup`, the default blue colour window cannot see a green cup. The agent calls `check_setup` and gets "target not seen". It then calls `resample_target_colour`, which finds the cup and applies the window, and the next reaches are verified. The rule coach alone cannot do this (§8.3).

Autonomy is a table in code, not a sentence in a prompt. Every tool is in one of four classes:

- `autonomous`: read-only checks, and speak.
- `patient_approval`: dose changes and resume. These become Yes / Keep / Why prompts. They are applied only on Yes, and clamped twice: by the prescription, then again by the pipeline.
- `forbidden_after_stop`.
- `never`.

Once the session has ended (feedback sent), only reads and the report note run. Other checks in code:

- Evidence ids must be whole numbers that exist in the session's own event ring. Every dose number must be finite. (Until the 11 Oct audit, a NaN clamped to the prescription's maximum.)
- Agent outputs are intents that the socket loop picks up. The loop re-checks the stop rules, so if a pain stop lands mid-run, the outputs are dropped (`agent_drop`).
- When the patient's socket closes, the run in progress is cancelled (traced `outcome: cancelled`) and nothing pending runs. The feedback checkpoint replaces stale ones and still runs after a stop, so the report note can explain the stop.

- `request_resume` runs its own camera check and offers nothing while any check fails.
- The patient's free-text check-in note reaches the model only as a capped data field with markup stripped. The check-in event the model can read carries typed values only. A prompt-injection scenario tests that the model cannot move the target.
- MCP routes answer only loopback clients that send no proxy header, or a per-process token. From the internet on AWS they return 403. We tested this through uvicorn's proxy-header middleware with a forged `X-Forwarded-For: 127.0.0.1`.

Observable. Every run writes these rows to the session trace:

- `agent_run` (trigger, model, transport, tokens, ms, first-action ms, outcome)
- `tool` rows (args, result summary, ms, and an `opencv` digest for the OpenCV tools: failing checks, hint, markers, homography error, target seen)
- agent `decide` rows naming their run and tool
- `agent_error`, `agent_drop` and `coach_note`

One JSON log line per run goes to stdout, and the counters are at `GET /api/agent/metrics`. On screen, the agent's status shows *thinking* / *idle* / *offline*. If the agent errors, times out or hits the daily cap, the header says "Coach offline · rule coach" and the rule coach carries on.

7. AWS deployment and COOL

7.1 What runs

Compute	EC2 `i-09863f70fa4854f3e`, **c8g.large (Graviton4, Neoverse-V2, 2 vCPU, 4 GiB)**, us-east-2, 60 GB gp3, Elastic IP
Image	AWS Marketplace **COOL AMI** `ami-005a9f90c70ba8d9b` (Cloud Optimized OpenCV for Graviton v3, Ubuntu 24.04); `cv2` 5.1.0-dev from `/opt/cool`, SVE + NEON baseline, KleidiCV 0.7.0 HAL
Service	the same FastAPI service under systemd. The venv comes from `uv.lock` **minus** the OpenCV wheel, so `import cv2` loads COOL. The install stops unless `cv2.__file__` is under `/opt/cool`
HTTPS	CloudFront `E3F5WICHF5D88D` → `d235zashbh751z.cloudfront.net` (HTTP→HTTPS, caching off, all viewer headers forwarded so WebSockets can upgrade)
Network	security group: port 8000 only from the CloudFront origin prefix list and one admin IP; SSH from that IP only; MCP routes return 403 from the internet
Cost and safety rails	$50/month budget with email alerts at 80 %, 100 % and forecast 100 %; Anthropic spend limit + daily agent-run cap; root MFA; no long-lived keys (CloudShell). Rebuilt 21 Oct in the Praxis-Team management account because the 11 Oct member account became unreachable, so the member-account SCPs no longer apply to this instance
Deploy	one idempotent command (safe to re-run): `infra/ec2/deploy.sh <host> [--agent]`

Checked on the instance. On the 21 Oct rebuild, `/health` through HTTPS reports `opencv_build {version 5.1.0-dev, cool: true, dnn: true, arch: aarch64}`, with the hand and pose models loaded and the agent ready. A replay ran through CloudFront, and 3 Claude agent runs completed.

On 11 Oct, on the previous instance (same COOL AMI, same c8g.large type, in the old account):

- the full service test suite, run against COOL's `cv2`, passed **634 / 636**. The two failures check `.gitignore` rules and need a git checkout, which the host does not have.

- a full Waiter – cup replay over `wss://` through CloudFront produced 517 frames, 31 reaches, 33 coach decisions and agent runs.
- the benchmark below was measured.

Neither was re-run on the rebuild.

7.2 COOL vs stock OpenCV on the same Graviton4

`benchmark/cool_bench.py` times Praxis's own per-frame stages on identical JPEG bytes from the two replay clips (640 × 480, q80). It runs them under COOL's `cv2` and under the pinned stock wheel (`opencv-python-headless==5.0.0.93`, NEON, KleidiCV 26.03), taking turns over 3 rounds × 5 passes on one c8g.large. `benchmark/summarize.py` writes the table from the raw JSON (`benchmark/results/cool_vs_stock_c8g.md`, `raw_v1/`). This run (v1) measures the code after the 11 Oct perception audit. That audit's brightness normalisation adds about 0.7 ms to the warp and about 2 ms to the cup stage. The first run (v0) is kept in `raw_v0/`.

p50 ms per frame	Stock 5.0.0	COOL 5.1.0-dev	Speed-up
Peg mask (HSV `inRange`)	0.481	0.304	**1.58×**
Peg blobs (`findContours` + moments)	0.576	0.395	**1.46×**
Holes (`HoughCircles`)	2.181	1.526	**1.43×**
Occupancy (masked `matchTemplate` × 9)	3.359	2.878	**1.17×**
Cup (warp + normalise + `inRange` + contours)	3.369	3.317	1.02×
`warpPerspective` + normalise	1.185	1.319	0.90×
JPEG decode	0.566	0.620	0.91×
ArUco + homography + solvePnP	1.893	2.192	0.86×
Full NHPT frame (classical)	7.690	7.452	**1.03×**
Full cup frame (classical)	5.216	5.573	0.94×
Hand model (`cv2.dnn`)	21.61	24.53	0.88×

Compute cost per 1,000 frames (hourly price divided by frames per hour at the measured p50, one stream at a time): full NHPT frame **$0.00017 stock vs $0.00019 COOL**, full cup frame $0.00012 vs $0.00014. COOL's AMI adds $0.01/h.

What this means. COOL is 1.2–1.6× faster on the stages that take most of the time on the NHPT board: thresholding, contours, Hough and template matching. It is slower on decode, ArUco, the warp and the DNN. That is because the stock aarch64 wheel already ships a newer KleidiCV, so it is a strong baseline, not a weak one. End to end, the classical NHPT frame is 3 % faster and the cup frame 6 % slower. Per frame, COOL costs slightly more once you count its AMI fee. Both builds process a classical frame in under 8 ms, well inside the 66–100 ms budget at 10–15 fps. We run COOL in production because it runs the whole workload on Graviton and speeds up the NHPT's heaviest classical stages, not because it is cheaper. Not measured yet: an x86 (c7i) reference, throughput with several sessions at once, and browser-to-cue latency through CloudFront.

7.3 Why this shape

A single container keeps raw frames inside one process. There is no queue, no object store and no second service that could leak video. Graviton suits the classical OpenCV workload (heavy on SIMD, no GPU needed), and COOL is OpenCV built for Graviton. CloudFront gives the HTTPS a browser requires before it will open the camera, with no certificate management on the instance.

7.4 Not on AWS yet

The agent calls Claude through the Anthropic API, from the instance just as from a laptop. Amazon Bedrock is not used (the rubric allows any model, and the `bedrock` mode was removed on 12 Oct). Logs go to stdout as JSON, ready for CloudWatch, but they are not shipped. The store is SQLite, with the schema DynamoDB would mirror. There is no CDK app; the deploy is a script.

7.5 Observability (built, not yet verified on AWS)

The service writes one JSON line for:

- each session start and end (exercise, live or replay, duration, verified and unverified reps, per-frame processing p50/p95, fps, event-to-cue p50/p95);
- each stop-rule action, therapist flag, perception loss and socket error;
- a heartbeat every 60 s (`services/session/praxis_session/ops.py`).

Agent runs already had their own lines. No frames, usernames, free text, pain scores or flag details are logged, and a test checks this.

With `PRAXIS_EMF=1`, set in the systemd unit, the same lines carry CloudWatch Embedded Metric Format blocks (namespace `Praxis`, dimensions Exercise and Mode). So CloudWatch pulls metrics straight from the logs, with no SDK and no new dependency. A pinned, checksummed CloudWatch agent (`deploy.sh --cloudwatch`) ships the log file to `/praxis/session` (30-day retention) and sends host memory, disk and CPU. A CloudShell script (`infra/ec2/cloudwatch_setup.sh`) creates the instance role, the log group, a `Praxis` dashboard, four alarms that email through SNS (heartbeat missing, CPU, status check, agent fallbacks) and saved Logs Insights queries. A test checks that the dashboard and alarms name only metrics the service emits.

None of this has run on the AWS account yet. We will update this section after the first live run. Expected cost is within the CloudWatch free tier, or under about $2 a month (`infra/ec2/README.md`).

8. Evaluation

A script in `evaluation/` or `benchmark/` regenerates every number below, and the results files record the host and settings. We never pool real and synthetic data.

8.1 Pipeline validation on synthetic renders (`evaluation/results/dataset_v0.md`)

These clips are sheets rendered with OpenCV, with perspective, noise and scrambled order, plus a synthetic 33-point landmark stream for the arm. They are scored at the **product defaults** the app ships (debounce 2, clear 3).

Exercise	Labelled reps	Rep F1 (P / R)	Timing / angle error
Nine-Hole Peg Test	9	1.00 (1.00 / 1.00)	NHPT total-time error 0.0 s (0.2 s before 11 Oct)
Waiter – cup	32	1.00 (1.00 / 1.00)	reach-time MAE 0 ms (200 ms before 11 Oct)
Arm out to the side	16	1.00 (1.00 / 1.00)	peak-angle MAE 0.77° (landmark stream, not the detector)

These numbers check the pipeline and state machines. They do not show real-world accuracy. No real lighting, hands, shadows or pegs are involved.

An earlier scorer used looser settings than the app ships and reported 1.0 / 0 ms. We withdrew those numbers. The timing errors dropped to zero on 11 Oct for a different reason: a bug fix, at the same product defaults. The debounce and the stationary check decide *whether* a peg is seated or a cup placed. But the event used to be stamped on the frame that confirmed it. So every NHPT total was late by (debounce – 1) frames and every reach by (clear – 1) frames, and this bias changed with the frame rate. Events are now stamped at the first frame of the confirmed run (`t_seated`, `t_placed`, `t_grasp`; `tests/test_timing_bias.py`). On these renders the labels line up with frames, so zero is expected by construction. Real error will come from real motion. Until 24 Oct the cup scored 0.984 (31 of 32): the last placement was lost because the clip ended before the hand cleared. Since the 24 Oct cup fix a placement counts once the cup is still, without waiting for the hand.

Robustness sweep (`evaluation/results/robustness_v0.md`, renders only). 107 conditions over both sheets: tilt, lens focal length and barrel distortion, in-plane rotation, distance, light, colour cast, shadows, noise, blur, JPEG, marker and grid occlusion by four skin tones, peg and cup colours, clutter, US Letter printing, paper curl, mirroring, a squashed 16:9 frame, and five combined scenes. After the fixes in §5:

- First fix on 74 / 81 NHPT and 83 / 91 cup conditions. The misses are heavy blur, markers under a hand or out of the frame, and the old Letter print.
- All holes correct on 61 / 74 calibrated NHPT conditions (69 before the 12 Oct height-aware occupancy in §8.1.1, 65 before the 24 Oct still-board rule). The sweep's pegs are flat discs, and past the 50° setup advice a flat disc on the next row sits where a standing peg's column is. The sweep scores one cold frame, so with a hand already over the grid the board is never still and one peg waits. In a session the pegs are seated before the hand comes over, and then they all stay seated for every skin tone.
- The cup found within 10 mm on 71 / 77. Misses: band colours outside the default window before tap-to-sample, and a black band. Since 24 Oct a hand gripping the cup (~22 % visible) no longer hides it.
- One phantom, on a sheet curled 20 mm.
- The setup rows give the right advice in 80 / 81 and 90 / 91 conditions (a 5 px defocus now gets the blur hint without the marker hint).
- Median target error 0.23 mm.

Not covered and still open: real exposure and glare, black pegs, and a curled sheet, which nothing flags (`docs/RISKS.md`). 3-D pegs and cups are in §8.1.1.

8.1.1 Physically based 3-D renders of the NHPT and Waiter – cup (`evaluation/results/synthetic3d_v0.md`)

Synthetic 3-D renders, not real-camera evidence. The team did not record itself, so the next-best evidence for the two sheet tasks is a Blender (Eevee) scene built from simple shapes in code (`evaluation/blender/`). It has:

- the printed page from the product's own sheet code;
- 32 mm pegs (6–9 mm) standing in a perforated board (3–12 mm) over the printed rings, and a dish beside the sheet;
- a tapered cup with a 19 mm blue tape ring and a handle;
- a skin-toned stand-in hand and forearm;
- four lights (daylight, dim, warm bulb, window side light), shadows, shiny paper and plastic, motion blur, sensor noise and JPEG q70;
- a laptop camera 20–50° from straight down, with yaw, roll and a 0.8–0.95 × width lens.

Each condition is framed the way setup leaves it (all four markers in view). Ground truth comes from the animation script. Marker occlusion is ray-cast per frame. 16 clips (10 NHPT × 9 pegs, 6 cups × 32 reaches, 9,167 frames) play through the unchanged replay pipeline at product defaults.

	Rep F1 (P / R) before → after	**Timing after**
Nine-Hole Peg Test (90 pegs)	0.41 (0.58 / 0.31) → **0.97** (0.94 / 1.0)	seat time +0.42 s bias, per-peg split MAE 0.23 s, NHPT total MAE 1.2 s (10 of 10 completed)
Waiter – cup (192 reaches)	0.21 (1.0 / 0.12) → **0.79** (0.95 / 0.68)	reach-time MAE 0.72 s (bias +0.64 s)

These are the 25 Oct numbers: we rendered all 16 clips again on 24 Oct and scored them on the build with the 24 and 25 Oct fixes below. On 24 Oct, before the shadow fix, the peg test was 0.90 (0.92 / 0.88, 8 of 10 completed, total MAE 2.0 s). On 12 Oct (after the four fixes listed next) they were NHPT 0.85 (0.85 / 0.86, 9 of 10 completed, total MAE 2.6 s) and cup 0.80 (1.0 / 0.67, reach-time MAE 0.77 s).

The renders found four product defects. We fixed each one test-first (`tests/test_parallax.py`, `tests/test_cup3d.py`, `tests/test_nhpt.py`):

1. *A seated peg read as empty.* Seen from 20–50°, a standing peg lands on the sheet plane as a streak. The streak starts past its hole (because of the board's thickness) and ends 10–40 mm away. Looking only at the hole disc, the hole read empty, the streak became a "free peg" (phantom grasps and drops), and at ~50° it lit up the next row's hole. Now occupancy looks first at a **seat column** for each hole: the hole raised 10–24 mm and projected with `cv2.projectPoints`. The camera model for that projection gets its focal length from a single-view `cv2.calibrateCamera` on the 16 marker corners (within 2 % of the true focal length on renders; the nominal 1.2 × width put a peg top 3–11 mm off). Free pegs exclude every hole's footprint.
2. *The forearm passing over seated pegs* flipped them to empty, and they were counted again. A hole now keeps its state while something covers it.
3. *A peg whose lift the hand model missed* was silently dropped, and the test never started. It is now counted as unverified (`grasp_not_seen`), and an inferred timer start is labelled.
4. *The cup was never found.* The blue ring is a vertical band. A laptop camera sees its near half, a crescent smaller than the 30 mm disc gate. The crescent's camera-side edge is the cup's bottom edge, which sits on the paper, so a circle fitted to it gives the centre of the base. The session learns the cup's radius from tight fits, and fits partly hidden bands for the centre only.

We did not loosen any threshold to raise a number.

What still fails, and why:

- The OpenCV Zoo hand model finds the simple stand-in hand on only 3–9 % of frames. So almost every peg is `unverified` (grasp or hand-near-hole not seen), and verified agreement means little here. Most inferred timer starts miss the first transfer (≈1.5 s).
- 6 extra NHPT reports (14 on 12 Oct). Most count an already seated hole again after the hand hid it, or read a peg carried over an empty hole as seated there. A hole counts once toward the nine, so they no longer end the test early.
- The dim-light clip with 6 mm yellow pegs, which never confirmed a seat on 12 Oct, now counts all nine (the board-change reference sees them).
- Fixed on 25 Oct: two clips lit by a low window (`nhpt3d_03`, `nhpt3d_07`) stopped counting at 4 and 5 of 9 on 24 Oct. Each standing peg cast a long shadow; the shadows joined into one hand-sized region, which was read as a hand covering the board, so the next pegs were blanked. Cast shadows (paper darker by the same factor in every channel) off the peg footprints no longer count toward a hand (`tests/test_peg_shadows.py`). That then showed a second bug: the test ended after nine seats even when one hole was counted three times, so it now ends when nine different holes are filled (`tests/test_nhpt.py`). Both clips now finish (25.3 s vs 26.8 s, 21.3 s vs 22.5 s).
- One cup clip (`cup3d_06`, 35°, yaw 14°) now counts dot 2 five extra times (5 extra reports, all verified). The cause is open; the 24 Oct rules that ignore up to two stray readings near a dot are the first suspect.
- Two cup clips fail. In dim light, a grey cup's noise reads as blue band. At 20°, the cup on an outer dot or the resting hand always hides a corner marker, so the four-marker first fix never comes (`docs/RISKS.md`).
- Every other cup clip loses the first one or two reaches the same way: the cup standing on dot 1, as the instructions say, hides the corner marker behind it.
- Reach times run long because the gripping hand hides the band at grasp and release.

24 Oct 2026, from the team's real sessions. Real use found three more defects. We fixed them test-first on synthetic frames:

1. *Uncoloured pegs.* The team's wooden, white and grey pegs never entered the colour mask (`pegs_detected: 0`). Now the board seen when the holes are first found becomes an empty reference. Every pixel that changed since then (exposure-matched `cv2.absdiff`, before paper normalisation) joins the peg mask. `cv2.connectedComponentsWithStats` sorts hand-sized changes outside the peg footprints into an occluder, which hides holes instead of filling them. Ghosts of things removed since are ignored (`tests/test_uncoloured_pegs.py`).
2. *An ordinary cup.* Only a blue tape ring was ever found, and nothing told the patient to use one. Now the sheet at the session's first fix is the reference, and a cup of any colour is whatever changed. Its base is fitted to the camera-side bottom edge of the body outline (`tests/test_cup_any_colour.py`).
3. *Tracking lost.* One real NHPT session had paused 19 times. Now, after the first fix, one or two markers sitting where the last fix puts them keep the homography for however long the hand or cup covers the rest (`source: held`), and two moved markers refit it. The first fix still needs all four markers, so the steps now keep the cup beside the sheet until the corners are found.

This is still unmeasured on real video. A stress run the same day (12 full nine-peg tests through `SessionPipeline` at product defaults: red, wooden, white and grey pegs, paper and wooden boards, 20–45° tilt, a forearm over the markers, a 20 % light drop) passed 11/12 after one more fix. Wood-grain specks in the colour mask (5–30 mm²) were read as loose pegs and held the FSM in `seated`. The colour mask is now opened at 1.7 mm, and a blob must be ≥ 25 mm² to count as a peg. The one remaining

failure is a limit of a single camera: a peg hovering 3–6 cm above a hole for ~0.5 s with no hand in view projects onto the hole in front of it.

The first real NHPT session after these fixes (team test, 24 Oct) counted all nine pegs in 30.9 s with no tracking loss. But the hand model missed the hand, so seven seats came back unverified (`grasp_not_seen`, `no_hand_near_hole`), and three in a row paused the timed test for 21 s, inside peg 7's split. The board saw those pegs go in, so these two reasons no longer count towards the `failed_reps` pause (`tests/test_stop_rules.py`).

The team's review of that session found the count itself was wrong: only one real peg went in, and the hand moving over the board made the other "pegs" (holes 3 and 6 were each counted twice, and three false `drop` events fired). Change from the empty board cannot tell a peg from fingertips or a hand's shadow on its own. Two rules now guard it (`tests/test_uncoloured_pegs.py`). A hole filled by change alone, with no peg colour, counts only after the board has been quiet for three frames in a row: no hand-sized change, and less than 60 mm² of change appearing or going since the last frame. A resting peg does not move; fingertips and shadows do. (A first version that blocked any change outside the peg footprints never let the board go quiet, because a spare peg beside the board is change too; the stress run caught it, 2/12.) Loose pegs (lifts and drops) now come from colour only, so with uncoloured pegs the timer start is inferred from the first seat unless the hand model sees the grasp; in the stress run that made the total 0.75 s short (14.0 s against 14.75 s). The same 12-run stress set passes 12/12 with these rules, including the hovering peg that failed before. The cup review found two more problems (`tests/test_cup.py`). Before the cup was on any dot, the screen already highlighted dot 2, so every count looked one dot ahead; it now highlights dot 1 until the cup is seen there. And the cup state flipped between "over a dot" and "moving" every 0.1 to 0.3 s, so placements took 5 to 23 s to count in three sessions (one went placed → moving → over the dot → placed 38 times) while clean ones counted in 0.5 to 1 s. Network and processing were not the cause (capture to event 132 to 147 ms median, processing 7 to 27 ms; the one 1.1 s p95 was a session run while screen sharing). Over the dot and once placed, up to two stray readings in a row are now ignored, and three (about 0.25 s) are needed to leave the dot. We could not reproduce the stray readings on synthetic frames (a hand gripping the cup kept the base within 1 mm), so their cause on real video is still open.

A deeper synthetic run followed the same day. For the NHPT it was 30 full tests: red, wooden, white-on-wood, grey and grey-on-wood pegs, each at 20°, 35° and 45° of camera tilt with two random seeds, with fingertips sweeping over the board and pauses between pegs. Fingertips held still for 0.5 s over empty holes are quiet, so the quiet rule alone let them through and gave a false split (timer 3.75 s short). So a hole filled by change alone also counts only when less than 30 % of it is stray change, meaning change components with more than 40 % of their area outside the peg footprints (fingers, shadows). This blocks new seats only; a seated peg the fingers touch stays seated. (Removing those pixels from the peg mask instead merged seated pegs with fingers and double-counted at 35° and 45°: 11 of the first 25 failed.) With both rules 29 of 30 pass with 9 pegs, no doubles and no drops, and the timer is within −0.75 to +2 s. The frozen-finger case now counts nothing extra (timer −0.75 s, the same inferred-start offset as clean runs). The one failure is white pegs on a wooden board at 45° (one hole counted twice, one missed). The paper normalisation tints the wood there, which is a known limit. For the cup it was 36 runs (white, red, green and blue-band cups at 30°, 40° and 50°, with 0 to 2 stray readings per placement). All 36 count the right eight dots in order with no wrong-dot or unverified reps, and each placement is counted at most 0.25 s after the cup is still with the hand clear. To get there, the "cup still" and "hand clear" waits now overlap instead of running one after the other, and stillness is measured from where the cup settled, not from the last reading. Arm out to the side was re-run on REHAB24-6 with no change (3.81° MAE, rep F1

0.931, pose found in 100 % of frames). The team's arm sessions with 0 verified reps were real short holds (0.3 to 1.2 s against the 3 s hold), not missed reps.

8.2 Latency (`evaluation/results/latency_v0.md`)

These runs drive the real WebSocket handler in-process the way the browser does (JPEG q80, capture-time prefix, ack-paced), on an Apple-silicon laptop with the hand model on. Pooled p50 / p95: **capture → event 9.7 / 14.8 ms**, **event → cue 0.8 / 1.2 ms** (rule coach). This does **not** include the JPEG encode in the browser, the network hop or rendering. The live screens show a browser-side p50/p95, which we will report from real sessions. The agent's first action takes 3.7 / 6.2 s on the synthetic scenarios and 3.7 / 5.8 s on real video (Sonnet 5.5, §8.3). That is why it works at checkpoints and never on in-rep cues.

8.3 Agent coach (`evaluation/results/agent_video_v0.md`, `agent_models_v0.md`, `agent_ablation_v0.md`)

`evaluation/scripts/agent_eval.py` runs 16 scenarios built from synthetic OpenCV frames, injected pose landmarks and scripted event streams. They cover:

- a clean session, covered marker, dim light, green cup, arm facing the wrong way, falling peaks;
- repeated lean (the patient declines the proposed change, and nothing changes);
- three unverified reaches, a pain stop mid-run, prompt injection, model timeout, MCP down;
- (11 Oct) the peg-test explanation with no clinical claims, the model API dropping mid-session, and a dose request outside the prescription (refused or clamped, and traced);
- the 16th, `mixed_signals` (21 Oct), a judgement scenario. Peaks slide 90→80°, and the last two lifts reach 90° only by leaning. A pass names both signals and gets a lower target accepted. The scripted and rule coaches fail it by design.

Each scenario is checked for task success, perception tool before action, guardrails, fallback and human approval.

Current headline (21 Oct, Claude Sonnet 5.5, the shipped default; `agent_recheck_2026-10-21_all.md`): 15 scenarios × 1 on the current build: **15 / 15** passed (95 % Wilson 0.80–1.0), **0 guardrail violations**, correct perception tool before acting 13 / 13, fallbacks 3 / 3, patient approval respected 8 / 8, first action p50 / p95 3.7 / 6.2 s, $0.020 per session. **Judgement scenario `mixed_signals` (21 Oct, Sonnet 5.5 × 3): 0 / 3.** Each run told the patient to stay upright and said the arm came back strong, but none offered a lower target (`docs/AGENT.md`). We report this as a miss and did not tune it away. The older runs below are kept for history.

Claude Sonnet 5, Anthropic API, 15 scenarios × 3 (11 Oct, final run after all audit fixes; `agent_v0_anthropic_sonnet-5_2026-10-11.md`)	
Runs passed	**45 / 45** (1.0, 95 % Wilson 0.921–1.0)
Guardrail violations	**0**
Correct perception tool before acting	39 / 39
Fallbacks (timeout, MCP down, outage mid-session) correct	9 / 9
Patient approval respected	24 / 24
Checkpoint → first action p50 / p95	4.8 / 8.2 s

Claude Sonnet 5, Anthropic API, 15 scenarios × 3 (11 Oct, final run after all audit fixes; `agent_v0_anthropic_sonnet-5_2026-10-11.md`)	
Cost	~$0.07 per session (list price; $3.25 for the 45 runs)

Earlier runs:

- An earlier run the same day passed 44 / 45. It came before the camera audit's fixes and before `covered_marker` was made realistic. Its one miss named the right fix but ran out of the 15 s run budget, and it failed safe (coach offline, rule coach carried on).
- The 10 Oct run (12 scenarios, 32/36) had three kinds of miss, all resolved. `mcp_down`'s check expected a spoken line on a clean arc, which was too literal (staying silent is allowed), so it now checks the fallback itself. `green_cup` once did not say why the cup was missed, so the prompt now asks the agent to name the colour as the cause. `falling_peaks` passes.
- Claude kept every dose request inside the prescription on its own, so only the scripted planner exercises the out-of-range clamp.
- A recheck on the evening of 12 Oct (Sonnet 5.5, × 1) gave 13 / 15 with no guardrail violation. Its two misses: a pause run that hit the 15 s run limit (`green_cup`, rule coach carried on), and a `dim_light` run that named the fix and safely never asked to resume, which the old check counted as a miss. Pause checkpoints now get 25 s (the session is already paused), and the check accepts a run that never asks to resume. Those two scenarios × 3 then passed 6 / 6 (`evaluation/results/agent_recheck_2026-10-12_{before,after}.md`).

Ablation: what the agent adds over the rule coach (same scenarios, agent off vs on):

Capability	Rule coach only	Agent (real Claude, first repeat)
Names the specific setup fix	1 / 6	**6 / 6**
Resumes only after a fresh camera check	0 / 5	**5 / 5**
Offers a dose adaptation (patient decides)	1 / 3	**3 / 3**
Recovers a non-blue cup (verified reaches after)	0 / 1	**1 / 1**
Session plan with evidence	0 / 2	**2 / 2**
Coach note in the report	0 / 1	**1 / 1**
Pain stop honoured, silent after	1 / 1	1 / 1
Peg-test time explained, no clinical claims	1 / 1	1 / 1
Unsafe actions	0	0

The scenarios are synthetic, and no patient has used the agent. This shows the tools, the guardrails and the model's tool choice working. It does not show clinical judgement.

On real video (`evaluation/results/agent_video_v0.md`, `evaluation/scripts/agent_video_eval.py`, 12 Oct). REHAB24-6 recordings (the dataset in §8.4) go frame by frame through the same pipeline, stop rules, rule coach and agent as the Replay socket. The patient's taps are simulated the same way for every planner. There are twenty cases:

- nine front-view sessions of two five-rep sets (the physiotherapist's correct block, then the deliberately wrong block). Five of these run on into the subject's real turn to half-profile.
- eight recordings that are side-on or half-profile from the first frame.

- three induced perturbations (frames cropped so the hips leave the view).

The ground truth, which the agent never sees, is the physiotherapist's label and motion-capture facts for each set. A shoulder hike is judged by the product's own angle-aware limit (`hike_allowance(angle)`, §8.4), not the fixed 0.08 used until 11 Oct. Re-run 12 Oct on pipeline `68a03c2a4fa5` (angle-aware hike, 0.55 front-view gate).

Real video, 2 repeats (Wilson 95 %)	Claude Sonnet 5.5	Claude Haiku 5.5	Scripted agent	Rule coach only
Faulty set reviews where a mocap-true fault was named and corrected	**41/44** (0.82–0.98)	41/44	0/22 (praise only)	22/22 (in-rep cues)
Claims supported by motion capture	67/83	62/83	n/a	36/79
Smaller dose offered, patient decides	19/44	27/44	0/22	0/22
Real wrong-view pause: fix named · no resume while still wrong	**26/26 · 26/26**	26/26 · 26/26	13/13 · 13/13	13/13 · 13/13 (pause text)
Induced hips-out: resumed after the fix (camera-checked for agents)	6/6	6/6	3/3	3/3
Guardrail violations	**0**	0	0	n/a
First action p50 / p95	3.7 / 5.8 s	3.4 / 6.4 s	n/a	n/a
$ per session (list price)	0.050	0.004	0	0

How to read this:

- Every set in this dataset is faulty under our prescription (the subjects lift overhead and rarely hold 3 s). So we measure false corrections as unsupported claims. 14 of Sonnet 5.5's 16 are "hold for the full count" where motion capture measured ≥ 2.5 s. Of its three misses, two gave the overshoot fix but sent it as praise, and one was a run timeout.
- The agent never named a fault the camera cannot measure (bent elbow, arm forward). Those are most of the physiotherapist's deliberate mistakes, so physio-wrong and physio-right sets get the same response.
- The rule coach corrects the same faults during the rep, but fewer of its claims hold up (36/79). It says "a touch higher" in sets where motion capture puts the lifts at 85° or more. Its opening "keep the shoulder down" counts as a hike claim, and motion capture rejects it in 13 sets.
- What the agent adds is the set-level dose proposal, the plan revision, corrections that cite the set's events (41/41 vs 15/22), and setup recovery checked by the camera.

Running real video found two product bugs, both fixed and tested. A pause from the same rule re-triggered the agent every 3 s while the subject stayed turned. And a pause 3 s after the opening line swallowed the agent's spoken fix (now a 4 s speak gap).

Model choice (`evaluation/results/agent_models_v0.md`). Our criteria, in order: task success, then cost, then latency, with 0 guardrail violations required. Synthetic 15 × 2:

- Sonnet 5.5: 28/30 (an earlier run 30/30), first action p50/p95 4.2/6.5 s, $0.019 per session
- Haiku 5.5: 30/30 (earlier 30/30), 3.2/5.3 s, $0.001 per session
- Sonnet 5: 29/30, 5.3/10.7 s, $0.018 per session

Haiku 5.5 was as accurate and far cheaper. But in both real-video runs before the hike re-tune, it once asked for a 100° target (refused by the clamp, counted as a violation). So the default is **Sonnet 5.5**. In the

re-run on the current pipeline Haiku had no violations (2 in 120 real-video sessions overall), which reopens the choice (`docs/AGENT.md` § Model choice). With caching an agent run costs ≈ $0.013. The daily cap of 100 runs limits spend to ≈ $1.30 a day.

8.4 Arm angle on real video (REHAB24-6) (`evaluation/results/rehab24_v0.md`)

REHAB24-6 [12] is a public rehabilitation dataset. It has RGB video from two synchronised 30 fps cameras, a 26-joint motion-capture skeleton, and repetitions that are segmented and labelled by a physiotherapist. Its exercise 1 is arm abduction: 13 recordings, 9 healthy adults, 178 repetitions.

`evaluation/scripts/rehab24_eval.py` plays every video through the **unchanged** `SessionPipeline` (Arm out to the side, right arm, every threshold at its product default; 15 fps, frames scaled to 640 px and JPEG-encoded like the browser's). It compares the product's angle with **anatomical abduction from motion capture**, measured in the subject's frontal plane (the flat plane facing forward through the body). Only the dose is raised, so counting does not stop at 16 reps. The licence is CC BY-NC 4.0: evaluation only, nothing redistributed.

Subject facing Camera17 (our front view): 88 reps, 8 subjects	
Angle error per frame (7,001 frames)	MAE **3.8°**, bias +0.7°, 95 % limits –9.1° to +10.5°
… inside the prescribed range (≤ 100°)	MAE 3.7°
Peak angle per rep	MAE **3.8°**, bias +0.2°
Reps counted by the state machine (verified or not)	F1 **0.93** (precision 0.87, recall 1.00)
Lifts found in the angle with the hold rule removed	F1 0.97
Pose found / view gate open	100 % / 100 % of frames
Correct reps flagged as a shoulder hike	**0 / 47** (47 / 47 with the fixed limit until 11 Oct)

These numbers are after the 12 Oct threshold changes below. Before them: 6,766 frames scored, rep-peak MAE 4.1°, F1 0.94, gate open 96.6 %. Side-on, the gate stays closed and asks the patient to turn (nothing is scored). At half-profile (labelled ~45°) it is now open on 34 % of frames (5 % before), and there the angle reads ~12° low. One-Euro smoothing adds no measurable lag at 15 fps.

What the real video changed. It exposed two defects in the pose stage. We fixed both test-first (`perception/pose.py`):

- The OpenCV Zoo port ran the person detector on every frame. When the arm went overhead, the detector put the mid-hip on the chest and the skeleton collapsed onto the face.
- It also cropped 1.25× tighter than MediaPipe does.

Praxis now tracks the region from the previous landmarks, as MediaPipe does, and crops at 1.25×. Before the fix, the same front-view data gave an angle MAE of 10.0° (limits –40° to +33°), a rep-peak MAE of 16.4° and a lift F1 of 0.38. A third defect was a crash: a region wholly outside the frame raised `cv2.error`, which would have ended a live session.

Two thresholds set from it (12 Oct, `evaluation/results/rehab24_thresholds_v0.md`).

Shoulder hike. The fixed limit (0.08 trunk lengths) fired below 90° on 47 of 88 healthy reps. That is because normal shoulder elevation reaches it at 80–95° of abduction (motion capture confirms this). The flag now fires only when elevation is above the healthy p95 *at the same arm angle* plus 0.05 trunk lengths

(~2.5 cm), held for 3 frames. We fitted the curve on four subjects. On the four held-out subjects, correct reps flagged fell from 22 / 22 to 0 / 22 (with the folds swapped: 25 / 25 to 0 / 25). A synthetic shrug still fires, but the dataset labels no shrugs, so **sensitivity on real shrugs is unmeasured**.

Front-view gate. The dataset only gives orientation as front / half-profile / side. So we recovered each subject's yaw (how far they are turned) to each camera, per frame, from the motion capture. We got each camera's projection matrix by DLT from the authors' projected joints, then compared the shoulder axis with the camera direction. Adults standing square-on sit at 0.61–0.70 trunk lengths of shoulder separation, so the 0.6 line passed only 36 % of frames at 20–30° of turn. At **0.55** it passes 100 % up to 20° and 93 % at 20–30°, where the angle error is still 6–8°. It rejects 97 % at 45–60° and every side-on frame. The cost: at 30–45° of turn, 69 % of frames pass, and the angle reads about 11° low there.

The head must be in frame. With the face masked, the pose was found on 51 % of frames and was wrong (MAE 26°). With the head cropped out, it was found on none. Our code never reads the face, but the MediaPipe detector uses it as an anchor.

Caveats. These are healthy adults standing in a motion-capture lab, filmed by corner cameras several metres away, wearing black suits. That is not a patient seated 2 m from a laptop. The dataset's lifts go overhead and never hold for 3 s, so only 16 of 100 counted reps were *verified*, which is the correct behaviour on this data. The reference is motion capture in the frontal plane, not a goniometer. NHPT and Waiter – cup have no real-data number yet.

8.5 End to end in a real browser with a simulated camera (`web/scripts/e2e-fake-camera.mjs`)

We launch Chrome with a fake webcam fed by videos from our renderer, made to look like a laptop camera (40° tilt, noise, the raw frame upside down). A DevTools-protocol script then walks the judge path against the real service:

- camera setup for both sheets, with a good clip plus four failure clips (covered corner, too dark, blurred, half out of frame);
- a full NHPT session;
- a full Waiter – cup session with the progression prompt;
- a sheet lost for 4 s mid-session;
- both replays;
- the arm setup screen with nobody in view.

The final run passed **76 / 76 checks**. Each failure clip got its own hint. The overlay landed within 1.1–1.4 px of the rendered sheet. The frames sent to the service were unmirrored. 9 / 9 pegs were counted once, with the timer within 0.2 s of the clip. 32 / 32 reaches were counted. The session paused and resumed by itself. Capture → event latency was 22–34 ms.

Getting there fixed nine defects, including a peg counted twice after a tracking gap, a database race on Home, and setup that stalled on an optional row. A run with the Claude agent on confirmed its lines, its OpenCV tool calls in the trace, and a resume that waits until the corners are visible again. The camera is still simulated. This shows the product works end to end in a browser. It does not show that it is accurate on real footage.

8.6 Our own recordings: protocol built, not run for this submission

`evaluation/PROTOCOL.md` defines **dataset v1**: ≥ 5 healthy adult volunteers, consent on paper, pseudonyms, clips deleted after 30 days, and 99 reps each, including scripted errors and a phone goniometer reading for each lift. A 25-minute team version is in `evaluation/QUICK_PILOT.md`. The tooling is built and tested end to end on stand-in clips:

- `record_session.py` refuses to write anywhere git would commit;
- `label_clip.py` labels on the replay clock;
- `eval.py` scores every metric and, with consent, saves frames from every miss as a failure case.

The team did not record its own sessions for this submission. So the real-video evidence is REHAB24-6, for Arm out to the side (§8.4). **The Nine-Hole Peg Test and Waiter – cup have no real-camera accuracy number.** Their evidence is the synthetic validation, the robustness sweep (§8.1), the 3-D renders (§8.1.1) and the browser end-to-end run (§8.5).

8.7 Usability

We will give the standard 10-item SUS (`evaluation/sus/`, scored by `evaluation/scripts/sus.py`) after one default day per participant, with a target mean ≥ 75. No responses have been collected, and no usability sessions were run for this submission.

A team test on the hosted review build did run (up to 21 Oct): 15 live sessions, 4 testers including automated agent-driven sessions. This was a team shake-down, not a usability study. Each tester's sessions are saved under a name. `/api/review` (`review.py`) summarises where sessions went wrong per exercise and tester, using only the stored OpenCV events and the screen-08 answers. The snapshot is `evaluation/results/team_test_2026-10-21.md`. Its counts led to the 21 Oct arm-gate and hike-cue changes (§5, §6.2). They are not a usability score. No therapist has used the build. Accessibility was audited against WCAG 2.1 AA (§9.4).

9. Responsible use

9.1 Safety lives in code

- The **prescription** (`prescription.py`, versioned) holds the dose, `allowed_adaptations` and stop rules. Every proposal from either coach is clamped by `prescription.clamp`, shown as a prompt, applied only on the patient's Yes, and clamped again by the pipeline. The 90° shoulder ceiling is enforced in the arm state machine.
- The service computes the **check-in pain tier**: 0–3 full dose; 4–6 gentle dose within the allowed adaptations; ≥ 7 a careful warning with a therapist flag. At 7 or more, rest today is advised, or the patient can choose Continue gently (`continue_gently`). That runs the gentle dose with every in-session stop rule still on (24 Oct).
- **Stop rules** run before either coach (§6.1). Nothing the agent does can override or shorten a stop. `end_session` can end a session but never extend one, and queued agent output is dropped when a stop lands.
- Every pause, rest and stop says why on screen. A rule pause asks the patient before resuming, and "Why?" repeats the reason with its evidence.

9.2 Privacy

- Video is processed in memory and dropped. We store only events, metrics, the decision trace and the session summary. The app has no recording feature. Evaluation clips exist only with signed consent and are deleted after 30 days.
- With the agent on, check-in answers and event summaries are sent to the model provider. Images never are.
- The face is never analysed for pain, emotion or identity.
- The hosted endpoint has no sign-in. We chose this so judging is easy. It holds sample data only and says so. Spend is capped by a daily agent-run limit and the Anthropic spend limit. AWS has a $50 budget with email alerts only (no budget action, no SCPs). Real patients would need Cognito sign-in first (`docs/RISKS.md`). Since 21 Oct, the two therapist writes (practice days, Mark reviewed) are refused to public visitors. They need `PRAXIS_THERAPIST_TOKEN`, and without it only a local request can write, so the hosted therapist view is read-only.

9.3 Human control and transparency

Take a rest (which pauses) · Pain / feel unwell · End session are on every live screen. The browser speaks the coach's cues, and they are always captioned. Audio is in Settings › Audio. It has been off by default since 24 Oct so the page never talks unasked; one tap in the demo, or raising the session volume slider, turns it on. Tracking lost pauses counting and clears itself when the camera sees the patient or the sheet again. A pause the patient asked for is resumed only by the patient. The rule coach and the agent are both labelled, and every agent line is captioned. Settings → *Show technical details* opens the full trace: OpenCV event → stop rule → decision → tool → action, with timestamps.

9.4 Accessibility

We audited against WCAG 2.1 AA (`docs/ACCESSIBILITY.md`). axe-core runs over every screen, dialog and in-session state, both in the test suite and in headless Chrome, in both themes. A keyboard-only test walks the whole critical path. The audit fixed 18 issues (landmarks, focus traps, alert roles, radio and tab keyboard behaviour, reduced motion). Colour is never the only carrier of meaning. Known gaps:

- some designer palette pairs miss 4.5:1 contrast (listed);
- the coach's cues are spoken (browser SpeechSynthesis, with an Audio switch and a volume slider) and always captioned, but we have not checked the spoken path with a screen reader running;
- no session with a screen-reader user has been run.

9.5 Attribution

GRASP is UBC / Heart & Stroke Foundation material. Praxis uses the exercise ideas only, with instructions in our own words and our own illustrations. The OpenCV Zoo models are Apache-2.0. Avatar motion comes from UI-PRMD (ODC PDDL) and IntelliRehabDS (CC BY 4.0). REHAB24-6 (CC BY-NC 4.0) is used for evaluation only and never redistributed. All are recorded in `docs/LICENSES.md`, each one entered before we integrated it.

10. Limitations and next steps

- **Real-world accuracy is partial.** Arm out to the side is measured on a public lab dataset of healthy adults (§8.4), not on patients or a laptop camera. NHPT and Waiter – cup are synthetic-

only: HSV ranges, Hough parameters, hand thresholds and their state machines are tuned on renders. Dataset v1 is next (§8.6).

- **Angles are camera-relative.** Abduction from the front view is an estimate in the image plane, not goniometry. It agrees with motion capture to 3.8° MAE when the subject faces the camera (§8.4) and reads ~12° low at 45°. It has not been compared with a goniometer.
- **Shoulder-hike flag.** It is now angle-aware and no longer fires on healthy REHAB24-6 lifts (0 / 22 held-out correct reps, §8.4). But its sensitivity to a real shrug is unmeasured: the curve comes from healthy standing adults, and only synthetic shrugs are tested. The front-view gate now scores a patient turned up to ~20° (and 93 % of frames at 20–30°), at the cost of an angle that reads low between 30° and 45° (`rehab24_thresholds_v0.md`). The head must also be in frame for the pose stage to work. In the 21 Oct team shake-down (laptop, seated; 4 testers including automated agent-driven sessions, `team_test_2026-10-21.md`), the hike flag still fired well below 90° of arm angle on most reps, where the REHAB24-6 allowance almost never fires. We don't know the cause (seated posture, trunk length from partly hidden hips, or real shrugs), because at that time the stored events kept only the angle. They now keep the hike value and its allowance. The threshold stays unchanged until we have those numbers.
- **Parallax.** The sheet homography assumes a flat plane. Since 12 Oct the NHPT models peg height (a seat column from a single-view camera calibration), and the cup is located by its base edge, which lies on the paper. Both are checked on 3-D renders only (§8.1.1). Both assume the standard 32 mm peg and a board up to ~12 mm, and the 50° tilt advice is still based on geometry alone. Hands above the sheet still project off (the pinch, hand-near-hole and hand-clear gates work in the sheet plane).
- **Session start on the cup sheet.** The cup standing on dot 1, as the instructions say, hides the corner marker behind it at laptop angles. The first fix needs all four markers (locked), so the first reach or two are lost. At ~20° the cup and the resting hand can hide a corner for the whole run (§8.1.1).
- **The printed NHPT is a timing aid.** Flat printed rings are not the recessed holes of a physical board, so the times are not those of the standardized clinical instrument.
- **Exertion is behavioural.** Hesitation is not computed yet, and the thresholds are unvalidated.
- **The agent** is evaluated on synthetic scenarios and on real video of healthy adults in a lab (REHAB24-6, simulated taps), not with patients. It takes 3–7 s to act (first action p50 3.7 s, p95 5.8 s on real video), and its judgement is not clinically validated.
- **AWS** is EC2 + COOL + CloudFront. S3 and CDK are not built, and CloudWatch shipping is built but not yet run on AWS (§7.5). The agent model is Claude via the Anthropic API, not Bedrock. COOL's gain depends on the stage: 1.03× on a full NHPT frame, 0.94× on a cup frame (§7.2).
- **Scope.** Three exercises, by decision. Rotations and other shoulder exercises are dormant in the code because one webcam cannot measure them honestly.
- **The pain stop level of 7** is a product decision and needs a clinician's sign-off before any pilot.

Next, in order: real recordings and dataset v1 → retune on failure cases → SUS with ≥ 5 participants and one or two therapist interviews → CloudWatch logs → video.

11. Reproducibility

What	Command
Service tests (937)	`cd services/session && uv sync && uv run pytest -q`

What	Command
Web tests (1019), lint, types	`cd web && pnpm install && pnpm test && pnpm lint && pnpm typecheck`
Run locally	`uv run python tools/fetch_models.py && uv run uvicorn praxis_session.main:app --port 8000` ; `pnpm dev`
Replay without a webcam	app: Settings → Demo and judging → Replay; or `uv run python -m praxis_session.replay`
Pipeline numbers (§8.1)	`uv run python ../../evaluation/scripts/eval.py`
3-D renders (§8.1.1)	`uv run python ../../evaluation/blender/make_dataset_3d.py` (Blender 5.2, ~35 min on a free laptop), then `uv run python ../../evaluation/scripts/eval_3d.py` (`--use-cache` rescores the committed events without the clips)
Latency (§8.2)	`uv run python ../../evaluation/scripts/latency.py --replay`
Real-video arm angle (§8.4)	`uv run python tools/fetch_rehab24.py` (≈ 0.55 GB, git-ignored), then `uv run python ../../evaluation/scripts/rehab24_eval.py` (~15 min on 4 cores); the hike curve and gate threshold from those runs: `uv run python ../../evaluation/scripts/rehab24_thresholds.py` (seconds)
Agent evaluation (§8.3)	`uv run python ../../evaluation/scripts/agent_eval.py --planner scripted` (free, CI); `--planner rule` (ablation); `--planner anthropic --repeat 3 --yes` (paid, needs a key)
COOL benchmark (§7.2)	`benchmark/cool_bench.py` on the c8g instance, once under COOL's Python and once in a venv with the stock wheel (method in `benchmark/results/cool_vs_stock_c8g.md`)
AWS deploy	`infra/ec2/deploy.sh <host> --agent` (`infra/ec2/README.md`)
Printable sheets	`uv run python tools/make_sheets.py` → `docs/sheets/*.pdf`

References

We checked figures against the PubMed abstract or the WHO page cited.

1. Feigin VL et al. World Stroke Organization (WSO): Global Stroke Fact Sheet 2022. *Int J Stroke* 2022;17(1):18–29. https://pubmed.ncbi.nlm.nih.gov/34986727/
2. Harris JE, Eng JJ, Miller WC, Dawson AS. A self-administered Graded Repetitive Arm Supplementary Program (GRASP) improves arm function during inpatient stroke rehabilitation: a multi-site randomized controlled trial. *Stroke* 2009;40(6):2123–8. https://pubmed.ncbi.nlm.nih.gov/19359633/
3. Kwakkel G et al. Probability of regaining dexterity in the flaccid upper limb: impact of severity of paresis and time since onset in acute stroke. *Stroke* 2003;34(9):2181–6. https://pubmed.ncbi.nlm.nih.gov/12907818/
4. Minagawa H et al. Prevalence of symptomatic and asymptomatic rotator cuff tears in the general population: from mass-screening in one village. *J Orthop* 2013;10(1):8–12. https://pubmed.ncbi.nlm.nih.gov/24403741/

5. Lang CE et al. Observation of amounts of movement practice provided during stroke rehabilitation. *Arch Phys Med Rehabil* 2009;90(10):1692–8. https://pubmed.ncbi.nlm.nih.gov/19801058/
6. Essery R, Geraghty AW, Kirby S, Yardley L. Predictors of adherence to home-based physical therapies: a systematic review. *Disabil Rehabil* 2017;39(6):519–534. https://pubmed.ncbi.nlm.nih.gov/27097761/
7. World Health Organization. Rehabilitation (fact sheet). https://www.who.int/news-room/fact-sheets/detail/rehabilitation
8. Cieza A et al. Global estimates of the need for rehabilitation based on the Global Burden of Disease study 2019. *Lancet* 2020;396(10267):2006–2017. https://pubmed.ncbi.nlm.nih.gov/33275908/
9. Mathiowetz V, Weber K, Kashman N, Volland G. Adult norms for the Nine Hole Peg Test of finger dexterity. *Occup Ther J Res* 1985;5:24–33.
10. Oxford Grice K et al. Adult norms for a commercially available Nine Hole Peg Test for finger dexterity. *Am J Occup Ther* 2003;57(5):570–3. https://pubmed.ncbi.nlm.nih.gov/14527120/
11. Chen HM et al. Test-retest reproducibility and smallest real difference of 5 hand function tests in patients with stroke. *Neurorehabil Neural Repair* 2009;23(5):435–40. https://pubmed.ncbi.nlm.nih.gov/19261767/
12. Černek A, Sedmidubsky J, Budikova P. REHAB24-6: Physical Therapy Dataset for Analyzing Pose Estimation Methods. In: *Similarity Search and Applications (SISAP 2024)*, Springer LNCS, 2024. https://doi.org/10.1007/978-3-031-75823-2_2. Data: Černek A et al., REHAB24-6, Zenodo 2024, https://doi.org/10.5281/zenodo.13305826 (CC BY-NC 4.0).

Software, models and datasets (OpenCV 5, OpenCV Zoo, Strands Agents, FastMCP, UI-PRMD, IntelliRehabDS, REHAB24-6, GRASP materials) with licences: `docs/LICENSES.md`.